

2026-05-28 - Zephyr Shell Subsystem

Goal

Use Zephyr's built-in Shell subsystem to add an interactive command-line interface to your firmware, accessible over UART or USB.

What you will learn

- How to enable and configure the Zephyr Shell
- How to define custom shell commands with arguments
- How to nest commands into subcommand trees
- How to use the shell over USB CDC-ACM

Overview

The Zephyr Shell gives you a fully featured CLI — command history, tab completion, help text — with minimal effort. It runs as a dedicated kernel thread and dispatches commands to your registered handlers.

prj.conf settings

```
CONFIG_SHELL=y
CONFIG_SHELL_BACKEND_UART=y
CONFIG_UART_CONSOLE=y
```

To use the shell over USB instead:

```
CONFIG_SHELL=y
CONFIG_SHELL_BACKEND_SERIAL=y
CONFIG_USB_DEVICE_STACK=y
CONFIG_USB_CDC_ACM=y
CONFIG_UART_LINE_CTRL=y
```

Registering a top-level command

```
#include <zephyr/shell/shell.h>

static int cmd_hello(const struct shell *sh, size_t argc, char **argv)
{
    shell_print(sh, "Hello from shell!");
    return 0;
}
```

```
SHELL_CMD_REGISTER(hello, NULL, "Print a greeting", cmd_hello);
```

Subcommands

```
SHELL_STATIC_SUBCMD_SET_CREATE(led_cmds,  
    SHELL_CMD(on, NULL, "Turn LED on", cmd_led_on),  
    SHELL_CMD(off, NULL, "Turn LED off", cmd_led_off),  
    SHELL_SUBCMD_SET_END  
);  
  
SHELL_CMD_REGISTER(led, &led_cmds, "LED control commands", NULL);
```

Usage: led on, led off

Reading arguments

```
static int cmd_set(const struct shell *sh, size_t argc, char **argv)  
{  
    if (argc < 2) {  
        shell_error(sh, "Usage: set <value>");  
        return -EINVAL;  
    }  
    int val = atoi(argv[1]);  
    shell_print(sh, "Value set to %d", val);  
    return 0;  
}
```

Printing from outside the shell

Use `shell_fprintf` with a saved shell pointer, or use `printk` — both reach the console.

Built-in shell commands

Zephyr's shell comes with built-in commands you get for free: - `help` — list all registered commands - `kernel uptime` — system uptime - `kernel reboot cold` — restart the device - `log` — control log levels at runtime

Why this matters

A shell interface is invaluable for calibration, diagnostics, and field testing. It lets you interact with a running device without rebuilding firmware.

Practice tasks

1. Register a ping command that replies pong.

2. Add a `led` command with `on` and `off` subcommands that control `led0`.
3. Add a `read temp` subcommand that prints a simulated temperature value.

Example folder

See `src/2026-05-28_Zephyr_Shell` for the day's code example.

Next topic

Tomorrow we read data from a real I2C sensor and log it using everything learned so far.